

Bloc DevOps

Séance 4 — Déploiement continu (CD) & pipeline bout-en-bout

Blue/green, canary, rolling updates, atelier



Au programme

PARTIE 1 — Stratégies de déploiement

- CI vs CD vs déploiement continu
- Rolling update
- Blue/green
- Canary

PARTIE 2 — Construire le pipeline CD

- Étendre la CI de la séance 2
- Build & push de l'image (séance 3)
- Le job deploy, smoke test, rollback
- Secrets & bonnes pratiques

PARTIE 3 — Bout-en-bout & atelier

- Le pipeline complet, de bout en bout
- Vérifier un déploiement, rollback manuel
- Atelier

Où en est on?

Trois briques posées, une quatrième aujourd'hui — la dernière avant l'observabilité

1

Git

Dépôt, branches, PR, protection de
branche (séance 1)

2

CI

Lint + tests automatiques à chaque push
(séance 2)

3

Docker

Image multi-stage optimisée, poussée
sur un registry (séance 3)

4

CD

Déploiement automatique après l'image
— aujourd'hui

La séance 5 branchera l'observabilité par-dessus tout ceci — la dernière brique technique avant la finalisation du projet.

CI, delivery continu, déploiement continu

Trois notions souvent confondues sous le même sigle "CD"

Intégration continue (CI)

- Chaque push déclenche automatiquement build + tests (séance 2)
- Vérifie que le code fonctionne — ne le déploie nulle part
- Le pipeline s'arrête après un ✓ ou un ✗

Continuous delivery vs deployment

- Delivery continu : l'artefact (l'image Docker) est toujours prêt à être déployé, mais la mise en production reste déclenchée manuellement
- Déploiement continu : chaque changement qui passe la CI est automatiquement déployé, sans validation humaine — c'est ce que construit cet atelier
- Le choix entre les deux est organisationnel autant que technique : quel niveau de confiance dans les tests automatisés ?

Le problème du déploiement "big bang"

Remplacer toutes les instances d'un coup — la méthode la plus simple, et la plus risquée

- Toutes les instances de l'ancienne version sont arrêtées, puis toutes les instances de la nouvelle version démarrent — un vrai instant de coupure (downtime)
- Si la nouvelle version a un bug, TOUS les utilisateurs sont impactés en même temps, sans avertissement progressif
- Le rollback signifie répéter l'opération en sens inverse — aussi risqué et aussi lent que le déploiement initial
- C'est le point de départ historique du sujet : les trois stratégies qui suivent existent précisément pour éviter ce scénario

Rolling update : le principe

Remplacement progressif des instances, une (ou quelques-unes) à la fois



À un instant donné, le service est un mélange d'anciennes et de nouvelles instances — c'est la nature même du rolling update.

Comment ça marche

- Une instance à la fois (ou par petits lots) est retirée du trafic, remplacée, vérifiée, puis remise dans le trafic — répété jusqu'à ce que toutes soient à jour
- C'est le comportement par défaut d'un Deployment Kubernetes (aperçu du bloc Cloud) ou d'un déploiement orchestré simple

Rolling update : avantages et limites

Avantages

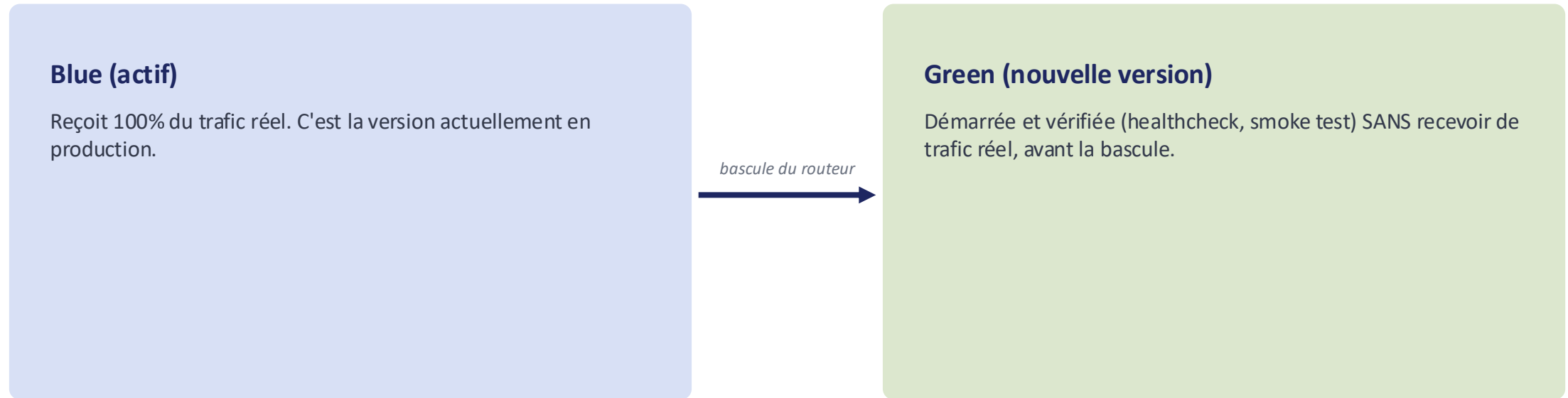
- Aucune infrastructure supplémentaire nécessaire — pas de doublement des ressources
- Aucune coupure de service pendant la mise à jour
- Le plus simple à mettre en œuvre des trois stratégies

Limites

- Deux versions du code tournent en même temps pendant la transition — nécessite une compatibilité ascendante (schéma de base de données, format d'API)
- Rollback plus lent : il faut re-dérouler le même processus en sens inverse, instance par instance
- Impossible de tester la nouvelle version sur un sous-ensemble contrôlé du trafic réel avant la généralisation

Blue/green : le principe

Deux environnements complets, un seul actif à la fois — bascule instantanée



- Une fois green vérifié sain, un reverse proxy ou un load balancer bascule instantanément 100% du trafic vers green — blue devient l'environnement inactif, conservé quelques instants pour un rollback immédiat

Blue/green : avantages et limites

Avantages

- Bascule quasi instantanée — coupure de service proche de zéro
- Rollback tout aussi instantané : re-basculer le routeur vers blue, qui n'a jamais été arrêté
- La nouvelle version est entièrement vérifiée AVANT de recevoir le moindre trafic réel

Limites

- Double les ressources nécessaires pendant la transition : deux environnements complets tournent en parallèle
- Une bascule 100%/0% ne teste jamais la nouvelle version sur un sous-ensemble réel avant la généralisation totale
- Les migrations de base de données restent le point délicat : blue et green doivent rester compatibles avec le même schéma pendant la transition

Canary : le principe

Déploiement progressif à une fraction croissante du trafic réel

Étape 1 — 5% du trafic vers la nouvelle version



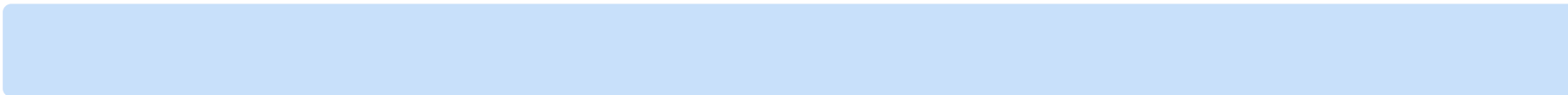
Étape 2 — 25% du trafic vers la nouvelle version



Étape 3 — 50% du trafic vers la nouvelle version



Étape 4 — 100% du trafic vers la nouvelle version



À chaque étape, on observe les métriques (taux d'erreur, latence — séance 5) avant d'augmenter la part de trafic. Un signal négatif interrompt la progression et rétablit 0% instantanément.

Canary : avantages et limites

Avantages

- Expose la nouvelle version à du trafic RÉEL, progressivement — le test le plus fiable qui existe
- Un problème n'impacte qu'une fraction des utilisateurs, pas tout le monde d'un coup
- S'articule naturellement avec l'observabilité: les métriques du canary pilotent la décision de continuer ou d'interrompre

Limites

- La stratégie la plus complexe à mettre en œuvre : nécessite un routage fin du trafic (pas juste 100%/0%)
- Nécessite une vraie observabilité en place pour être pilotée correctement — sinon on "vole à l'aveugle"
- Plus lente qu'un blue/green pour atteindre 100% — un compromis délibéré entre rapidité et prudence

Comparaison des trois stratégies

Stratégie	Risque	Complexité	Rollback	Coût infra
Rolling update	Moyen	Faible	Lent (progressif)	Aucun surcoût
Blue/green	Faible	Moyenne	Instantané	Double pendant la transition
Canary	Très faible	Élevée	Rapide (partiel)	Modéré, progressif

Quelle stratégie pour quel contexte ?

Aucune n'est universellement "la meilleure"

- Petite équipe, infra simple, peu de risque acceptable pour une coupure brève : rolling update suffit souvent
- Application critique, besoin d'un rollback quasi instantané, ressources doublées acceptables : blue/green
- Base d'utilisateurs large, changement risqué (nouvelle logique métier), observabilité déjà en place : canary
- Dans cet atelier, le programme demande une stratégie "même simplifiée" adaptée à l'environnement du groupe — l'important est de savoir justifier le choix, pas de suivre une recette unique
- Les trois stratégies partagent le même principe fondateur : ne jamais envoyer 100% du trafic vers du code non vérifié

Partie 2

Construire le pipeline CD

Étendre la CI de la séance 2 avec le build de l'image (séance 3) puis un déploiement automatique.

Le point de départ : deux jobs déjà en place

lint et test, hérités tels quels de la séance 2



```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [ ... flake8 ... ]
  test:
    runs-on: ubuntu-latest
    needs: lint
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    steps: [ ... pytest, cache pip, upload-artifact ... ]
```

Étendre le pipeline : build-and-push

Le Dockerfile multi-stage de la séance 3 entre en jeu



```
build-and-push:
  runs-on: ubuntu-latest
  needs: test
  permissions:
    contents: read
    packages: write
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
    - uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: ghcr.io/${ github.repository }:${ github.sha }
```

Authentication au registry depuis la CI

GITHUB_TOKEN : un secret déjà là, sans rien configurer

GITHUB_TOKEN

- Injecté automatiquement par GitHub Actions dans chaque run — aucune création manuelle nécessaire
- Sur ghcr.io, il suffit avec la permission packages: write déclarée dans le job
- Sa portée est limitée au dépôt et expire à la fin du run — surface d'exposition minimale

Docker Hub ou autre registry

- Nécessite un vrai secret créé manuellement : Settings → Secrets and variables → Actions
- Jamais le mot de passe du compte en clair — un token d'accès dédié, avec le droit push uniquement
- Même principe que les secrets CI déjà vus en séance 2 : secrets.NOM_DU_SECRET, jamais en dur dans le YAML

Stratégie de tag d'image

Un tag mutable ne suffit jamais seul en déploiement continu

`${{ github.sha }}`

Immuable, traçable jusqu'au commit exact — le tag à déployer réellement

`latest`

Pratique pour un usage manuel, mais mute à chaque push — jamais fiable pour un rollback précis

`v1.2.0 (tag Git)`

Utile pour marquer une release explicite — complète le SHA, ne le remplace pas

Le job deploy : anatomie

Cinq étapes, dans cet ordre précis — jamais de trafic vers du code non vérifié

1 Démarrer

La nouvelle version démarre dans l'environnement inactif — sans recevoir de trafic réel

2 Healthcheck

Attendre que le conteneur réponde (ex : /health) avant toute autre étape

3 Smoke test

Vérifier un comportement applicatif réel (pas juste "le port répond") — ex : /status renvoie le bon contenu

4 Basculer

Si tout est vert : le routeur (nginx, load balancer) redirige le trafic vers la nouvelle version

5 Nettoyer / Rollback

Si succès : arrêter l'ancienne version. Si échec à une étape précédente : arrêter la nouvelle, ne rien basculer

docker-compose.deploy.yml

Deux services identiques, chacun activable indépendamment via un profil



```
services:
  app-blue:
    build: { context: ., dockerfile: Dockerfile }
    profiles: ["blue"]
    ports: ["5001:5000"]
    environment: { REDIS_HOST: redis, DEPLOY_COLOR: blue }
  app-green:
    build: { context: ., dockerfile: Dockerfile }
    profiles: ["green"]
    ports: ["5002:5000"]
    environment: { REDIS_HOST: redis, DEPLOY_COLOR: green }

# docker compose --profile green up -d app-green
# ne démarre QUE app-green - app-blue reste inchangé
```

Attendre qu'un conteneur soit prêt

"Démarré" et "prêt à recevoir du trafic" ne sont pas la même chose



```
READY=0
for _ in $(seq 1 15); do
  if curl -sf http://localhost:5002/health \
    > /dev/null 2>&1; then
    READY=1
    break
  fi
  sleep 2
done
```

Pourquoi une boucle, pas un seul essai

- docker compose up -d rend la main dès que le conteneur est créé — pas quand l'application a fini de démarrer
- Un délai de grâce (ici 30s max) laisse le temps à l'application de s'initialiser (connexion à Redis, chargement de configuration...)
- Sans cette boucle, un healthcheck trop pressé déclencherait un rollback sur un déploiement qui aurait pourtant fini par réussir

L'environnement cible de cet atelier

Une simplification assumée et documentée — pas un raccourci caché

En production réelle

- L'hôte cible est un serveur persistant (VM, cluster) — le job deploy s'y connecte (SSH, API cloud, kubectl...)
- Les deux versions (blue/green) tournent sur cet hôte, en dehors du cycle de vie du runner CI

Dans cet atelier

- Le programme autorise explicitement un "environnement simple — conteneur local" — c'est le runner CI lui-même qui joue ce rôle
- Le mécanisme (deux versions démarrées, vérification avant bascule, rollback automatique) est rigoureusement identique à un vrai hôte de production
- Seule différence : l'hôte est éphémère (le runner disparaît à la fin du job) — en vraie prod, il persisterait

Smoke test avant bascule de trafic

Un healthcheck vérifie que ça répond — un smoke test vérifie que c'est CORRECT



```
# healthcheck : le port répond-il ?  
curl -sf http://localhost:5002/health  
  
# smoke test : la réponse est-elle  
  
# celle qu'on attend vraiment ?  
curl -sf http://localhost:5002/status \  
| python3 -c \  
"import sys, json; d = json.load(sys.stdin); \  
sys.exit(0 if d['deploy_color']=='green' else 1) "
```

Pourquoi les deux comptent

- Un conteneur peut démarrer et répondre sur son port tout en étant mal configuré (mauvaise variable d'environnement, mauvaise version déployée)
- Le smoke test de cet atelier vérifie que /status renvoie bien le deploy_color attendu — la preuve que c'est la bonne version qui répond
- En conditions réelles, un smoke test couvre aussi les dépendances critiques (la base de données répond-elle vraiment depuis cette nouvelle version ?)

Rollback automatique en cas d'échec

Le trafic ne bascule QUE si toutes les vérifications précédentes sont vertes



```
if [[ "$READY" -ne 1 ]]; then
    echo "ÉCHEC : app-$IDLE ne répond pas sur /health"
    echo "ROLLBACK : arrêt de app-$IDLE, $ACTIVE reste actif"
    docker compose --profile "$IDLE" stop "app-$IDLE"
    exit 1
fi

# le trafic n'a jamais été touché – $ACTIVE est resté actif
# du début à la fin de cette tentative de déploiement
```

Secrets de déploiement & environnements GitHub

Protéger l'étape la plus sensible du pipeline

Secrets propres au déploiement

- Clé SSH vers l'hôte cible, token d'API cloud, identifiants de base de données de production
- Toujours dans secrets.NOM_DU_SECRET, jamais en dur — même principe que la séance 2, appliqué à un contexte plus sensible
- Une fuite ici a un impact direct sur l'environnement de production, pas seulement sur le code

GitHub Environments

- environment: production dans le job déclare un environnement dédié, avec ses propres secrets et règles
- Peut exiger une validation humaine ("required reviewers") avant que le job ne s'exécute — un vrai garde-fou pour la prod
- Angle cyber du bloc : c'est le même principe que la protection de branche de la séance 1, appliqué au déploiement plutôt qu'au merge

Bonnes pratiques d'un pipeline CD

À viser

- Toujours vérifier (healthcheck + smoke test) avant de basculer le trafic — jamais après
- Un rollback automatique et scripté, pas une procédure manuelle à improviser sous pression
- Des tags d'image immuables (SHA), jamais latest seul en production
- Un déploiement idempotent : rejouer deploy.sh sans changement ne doit rien casser

À éviter

- Déployer directement sur 100% du trafic sans étape de vérification intermédiaire
- Des secrets de déploiement partagés entre environnements de test et de production
- Un script de déploiement qui ne gère pas explicitement son propre échec (pas de exit 1, pas de rollback)
- Confondre "le conteneur a démarré" avec "le déploiement a réussi"

Partie 3

Bout-en-bout & atelier

Le pipeline complet, du push au déploiement, avant l'atelier.

Le pipeline complet, vue d'ensemble

Un seul git push déclenche tout, sans intervention manuelle

- | | | |
|---|-----------------------------|---|
| 1 | <code>git push</code> | Le déclencheur — un push sur main ou une pull request (séance 2) |
| 2 | <code>lint + test</code> | Qualité du code et comportement applicatif vérifiés (séance 2) |
| 3 | <code>build-and-push</code> | Image Docker construite et poussée sur le registry (séance 3 + aujourd'hui) |
| 4 | <code>deploy</code> | Nouvelle version démarrée, vérifiée, trafic basculé — ou rollback automatique (aujourd'hui) |
| 5 | <code>Production</code> | L'utilisateur final sert désormais la nouvelle version, sans avoir vu de coupure |

Exemple de job deploy complet

Tel qu'il apparaît dans la solution de cet atelier



```
deploy:
  runs-on: ubuntu-latest
  needs: build-and-push
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with: { registry: ghcr.io, username: ${ github.actor },
              password: ${ secrets.GITHUB_TOKEN } }
    - name: Déployer (blue/green)
      run: ./deploy/deploy.sh
```

Vérifier un déploiement après coup

Trois commandes pour confirmer que tout s'est bien passé



```
$ curl http://localhost:8080/status
{"service": "projet-devops-groupe-demo", "deploy_color": "green"}

# confirme quelle couleur sert réellement le trafic
$ cat deploy/active_color
green

# état persisté, source de vérité pour le prochain déploiement
$ docker compose -f docker-compose.deploy.yml ps

# app-green (Up), app-blue (arrêté), nginx (Up), redis (healthy)
```

Rollback manuel après coup

Que faire si un problème n'apparaît qu'APRÈS la bascule automatique ?



```
# le rollback automatique de deploy.sh  
  
# protège avant la bascule — pas après  
$ git revert <commit-problématique>  
$ git push origin main  
  
# le pipeline complet se redéclenche,  
# construit l'image du commit précédent,  
# et la redéploie normalement
```

Pourquoi git revert, pas un rollback manuel

- Le dépôt Git reste la seule source de vérité — aucune divergence entre ce qui est en production et ce que raconte l'historique
- Le commit de revert repasse par tout le pipeline (lint, tests, build, deploy) — pas de raccourci qui contournerait les vérifications
- Un rollback manuel direct sur l'hôte de production (sans passer par Git) casse la traçabilité — à éviter sauf urgence absolue documentée après coup

Ce que ce pipeline ne voit pas encore

Un déploiement "réussi" techniquement peut rester un problème silencieux

- Le smoke test de cet atelier vérifie un instant précis, juste après le déploiement — pas le comportement dans la durée
- Une nouvelle version peut démarrer sainement puis dégrader progressivement (fuite mémoire, hausse du taux d'erreur, latence qui grimpe) sans que rien dans ce pipeline ne le détecte
- C'est exactement le rôle de l'observabilité: métriques continues, dashboard, et une alerte qui se déclenche si le taux d'erreur dépasse un seuil APRÈS le déploiement — pas seulement au moment du smoke test
- Un pipeline canary avancé (aperçu de la partie 1) piloterait même sa propre progression à partir de ces métriques

Lien avec le Cloud & IaC

Où s'arrête ce pipeline, où commence l'infra cible

DevOps

- Construit l'image Docker et la pousse sur un registry
- Définit COMMENT déployer (stratégie, vérifications, rollback)
- Dans cet atelier, la cible reste simplifiée

Plus loin..

- Fournit une vraie cible de déploiement persistante — un cluster
- kubectl apply ou Helm y jouerait le même rôle que deploy.sh ici : démarrer la nouvelle version, vérifier, basculer
- Kubernetes gère nativement le rolling update (et, avec des outils comme Argo Rollouts, le blue/green et le canary)

Sécurité du pipeline de déploiement

Le déploiement est l'étape la plus sensible de tout le pipeline

Principes

- Principe du moindre privilège : le token de déploiement n'a accès qu'à ce qui est strictement nécessaire
- Un GitHub Environment avec required reviewers ajoute une validation humaine avant un déploiement en production
- Les logs de déploiement ne doivent jamais révéler un secret (attention aux echo de variables sensibles)

Supply chain

- L'image déployée doit être celle scannée (docker scout / trivy) — jamais une image non vérifiée
- Le tag SHA immuable garantit qu'on déploie exactement ce qui a été testé, sans substitution possible entre-temps
- Cohérent avec l'angle cybersécurité du programme : chaque étape du pipeline est un rempart, pas seulement la revue de code initiale

Debugger un déploiement qui échoue

Les réflexes avant de soupçonner le script lui-même

- Le job deploy échoue au healthcheck : `docker compose logs app-<couleur>` — presque toujours l'erreur exacte y figure directement
- Le smoke test échoue alors que `/health` répond : comparer manuellement la réponse de `/status` à ce que le script attend — souvent une variable d'environnement mal transmise
- nginx ne bascule pas malgré un script qui semble réussir : vérifier que le fichier `active.conf` a bien été réécrit, et que `nginx -s reload` a réellement été exécuté (pas juste préparé)
- Le déploiement précédent semble "coincé" entre deux couleurs : lire `deploy/active_color` en premier réflexe — c'est la source de vérité de l'état courant
- Un smoke test qui passe en local mais échoue en CI : vérifier que le port testé correspond bien à l'environnement du runner, pas à un port local resté en mémoire

Récap — toute la syntaxe de la séance

```
build-and-push:
  needs: test
  uses: docker/login-action@v3
  uses: docker/build-push-action@v5
  tags: ...${{ github.sha }}
```

```
deploy:
  needs: build-and-push
  environment: production
  run: ./deploy/deploy.sh
```

```
docker compose -f *.deploy.yml
  --profile <couleur> up -d
curl -sf ../health
nginx -s reload
git revert <sha> && git push
```

Où s'arrête cette séance

Déployer n'est pas encore superviser

Aujourd'hui

- Le pipeline devient complet : push → lint → test → image → déploiement
- Une stratégie de déploiement (blue/green dans la solution) vérifie avant de basculer, et fait un rollback automatique en cas d'échec
- Rien ne surveille encore le comportement de l'application UNE FOIS déployée et en fonctionnement normal

Observabilité

- L'application expose des métriques (requêtes, latence, erreurs)
- Prometheus les collecte, Grafana les visualise dans un dashboard
- Une règle d'alerting prévient si quelque chose tourne mal après le déploiement — le maillon manquant identifié à la slide 30

Construire le pipeline CI/CD bout-en-bout

Rappel du pipeline &
choix de stratégie

Job build-and-push

Environnement
blue/green local

Script de déploiement &
rollback

Job deploy dans la CI

Test de bout en bout
(push réel)

Rollback manuel &
vérifications

Consolidation & README

Livrable de fin de séance

Un pipeline CI/CD de bout en bout démontrable en une seule commande (git push) : lint → tests → build de l'image → déploiement automatique avec une stratégie choisie , vérification avant bascule, et rollback automatique en cas d'échec.

Objectifs du jour

Pipeline bout-en-bout

Un seul git push déclenche build → test → image → déploiement, sans intervention manuelle

Stratégie de déploiement

Choix justifié et réellement mis en œuvre — vérification avant bascule, pas juste un docker run

Rollback

Automatique en cas d'échec de vérification, et procédure manuelle documentée (git revert) en cas de problème après coup

Intégration

Cohérent avec les étapes précédentes — pas un pipeline reconstruit de zéro

À retenir



CI vérifie le code, déploiement continu le met automatiquement en production — deux étapes distinctes



Trois stratégies (rolling, blue/green, canary) partagent un même principe : ne jamais exposer 100% du trafic à du code non vérifié



Un job deploy suit toujours le même ordre : démarrer → vérifier → basculer → nettoyer, avec rollback à chaque étape qui échoue



Un tag d'image immuable (SHA), jamais latest seul, garantit qu'on déploie exactement ce qui a été testé



Le pipeline complet reste muet sur le comportement APRÈS le déploiement — c'est le rôle de l'observabilité

Prochaine séance

Observabilité : logs, métriques, Prometheus & Grafana — surveiller ce que ce pipeline vient de déployer.